Day 140 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 21, Day 1 reference

Watch: Diffusion Models: DDPM | Generative AI Animated (Deepia) | Diffusion Models | Paper Explanation | Math Explained (Outlier)

`day140_reverse_diffusion.py`, segment by segment, in the order `main()` uses things. The posterior, the exact denoiser, the reverse sampling loop, the time embedding, the noise-predicting network and the spread measure are new; the noise schedule, q_sample, q_step, the ring and the mode count are Day 139's parts, and the network code and Adam are Day 126's, all imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: t = step (1–1,000) · β_t = BETAS[t – 1] · A_t = ABAR[t] = $\prod(1-\beta)$ · ε = noise · C_j = ring centre j

Segment 0 — Setup

Imports from two weeks back and one

CODE

```
for w in ("week-20-generative-models-gans", "week-19-unsupervised-learning-ii",
    "week-18-unsupervised-learning-i", "week-17-classical-ml-kernels-ensembles"):
    sys.path.insert(0, os.path.join(HERE, "..", w))
with contextlib.redirect_stdout(io.StringIO()):
    import day126_exercise as d126
    import day139_gan_build_and_forward_diffusion as d139
```

- `os.path.join(HERE, "..", w)` — ".." is the parent folder: today's script lives in a new week's folder and reaches back into earlier ones.
- `import day139_gan_build_and_forward_diffusion as d139` — Day 139's build file holds the schedule and every earlier part it was assembled from.

Segment 1 — The reverse process

The schedule, and the posterior

MATH

$$c_0 = \sqrt{A_{t-1}}\,\beta_t / (1-A_t), \quad c_t = \sqrt{(1-\beta_t)}(1-A_{t-1}) / (1-A_t), \quad variance = (1-A_{t-1}) / (1-A_t)\,\beta_t$$

CODE

```
BETAS, ABAR = d139.schedule("linear")
def posterior(t):
    b, ab, ab1 = BETAS[t - 1], ABAR[t], ABAR[t - 1]
    c0 = np.sqrt(ab1) * b / (1 - ab)
    ct = np.sqrt(1 - b) * (1 - ab1) / (1 - ab)
    return c0, ct, (1 - ab1) / (1 - ab) * b
```

- `BETAS, ABAR = d139.schedule("linear")` — tuple unpacking: the function returns two arrays. BETAS has 1,000 entries (index t – 1); ABAR has 1,001 (ABAR[0] = 1).
- `BETAS[t - 1], ABAR[t], ABAR[t - 1]` — three values in one line, unpacked into b, ab and ab1.
- `return c0, ct, (...)` — returns a tuple; callers unpack it or index it, as posterior(t)[2] does for the variance.

exact_x0: the best possible guess on the ring

MATH

$$p(j \mid x_t) \propto exp(-|x_t - \sqrt{A_t}\, C_j|^2 / 2v), \quad v = A_t\, 0.05^2 + 1 - A_t$$

CODE

```
def exact_x0(xt, t, radius=2.0, sd=0.05, k=8):
    C = np.c_[np.cos(2 * np.pi * np.arange(k) / k), np.sin(2 * np.pi * np.arange(k) / k)] * radius
    a, v = np.sqrt(ABAR[t]), ABAR[t] * sd ** 2 + 1 - ABAR[t]
    L = -((xt[:, None, :] - a * C[None]) ** 2).sum(-1) / (2 * v)
    w = np.exp(L - L.max(1, keepdims=True)); w /= w.sum(1, keepdims=True)
    m = C[None] + (a * sd ** 2 / v) * (xt[:, None, :] - a * C[None])
    return (w[..., None] * m).sum(1)
```

- `np.c_[np.cos(...), np.sin(...)] * radius` — np.c_ joins columns: eight (x, y) centres on a circle of radius 2.
- `xt[:, None, :] - a * C[None]` — shapes (n, 1, 2) and (1, 8, 2) broadcast to (n, 8, 2): every point against every centre.
- `.sum(-1)` — -1 is the last axis: add the x and y parts of each squared distance.
- `np.exp(L - L.max(1, keepdims=True))` — subtract each row's largest value before exp, so nothing overflows; the weights are unchanged after normalising.
- `w[..., None] * m` — ... stands for every leading axis: the weights (n, 8) become (n, 8, 1) to multiply the (n, 8, 2) means.

eps_from_x0 and reverse_sample

MATH

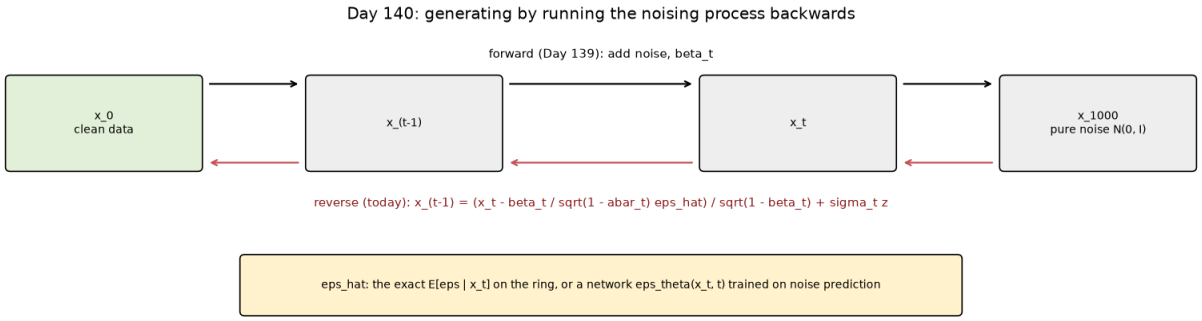
$$x_{t-1} = (x_t - \beta_t / \sqrt{(1 - A_t)} \cdot \varepsilon') / \sqrt{(1 - \beta_t)} + \sigma_t\, z$$

CODE

```
def eps_from_x0(xt, x0, t):
    return (xt - np.sqrt(ABAR[t]) * x0) / np.sqrt(1 - ABAR[t])
def reverse_sample(eps_fn, n, rng, var="posterior", d=2):
    x = rng.standard_normal((n, d))
    for t in range(len(BETAS), 0, -1):
        b = BETAS[t - 1]; e = eps_fn(x, t)
        x = (x - b / np.sqrt(1 - ABAR[t]) * e) / np.sqrt(1 - b)
        if t > 1:
            s2 = posterior(t)[2] if var == "posterior" else (b if var == "beta" else 0.0)
            x = x + np.sqrt(s2) * rng.standard_normal(x.shape)
    return x
```

- `for t in range(len(BETAS), 0, -1)` — counts down: 1000, 999, ..., 1 (the stop value 0 is not included).
- `e = eps_fn(x, t)` — eps_fn is a function passed in: the exact denoiser or the network, so one loop serves both.
- `if t > 1:` — the last step returns the mean only, as in Ho et al. (2020).
- `a if var == "posterior" else (b if var == "beta" else 0.0)` — two conditional expressions nested: three choices in one line.

SYSTEM DIAGRAM



Forward and reverse.

Segment 2 — The learned denoiser

time_embed: a step number as 32 numbers

MATH

$$embed(t) = [\sin(t\,f_0), \ldots, \sin(t\,f_{15}), \cos(t\,f_0), \ldots, \cos(t\,f_{15})], \quad f_i = 10000^{-i/16}$$

CODE

```
def time_embed(t, n=16):
    f = 10000.0 ** (-np.arange(n) / n); a = np.asarray(t, np.float64)[:, None] * f[None]
    return np.concatenate([np.sin(a), np.cos(a)], 1)
```

- `10000.0 ** (-np.arange(n) / n)` — 16 frequencies spaced evenly on a log scale, from 1 down to about 1/10,000.
- `np.asarray(t, np.float64)[:, None] * f[None]` — an outer product by broadcasting: every step times every frequency, shape (n, 16).

eps_net_fn: a network wrapped as a function

CODE

```
def eps_net_fn(net):
    acts = ["relu"] * (len(net) - 1) + ["linear"]
    return lambda x, t: d126.forward(net, np.concatenate([x, time_embed(np.full(len(x), t))],
        1).astype(np.float32), acts)[-1].astype(np.float64)
```

- `["relu"] * (len(net) - 1) + ["linear"]` — list repetition and joining: ReLU on every hidden layer, nothing on the output.
- `lambda x, t: ...` — returns a function with the same signature as the exact denoiser, so reverse_sample can use either.
- `np.full(len(x), t)` — the same step number for every point in the batch.
- `[-1]` — Day 126's forward returns every layer's activations; the last is the output.

train_denoiser: the noise-prediction loss

MATH

$$loss = mean\ |\varepsilon - \varepsilon_\theta(\sqrt{A_t}\, x_0 + \sqrt{(1 - A_t)}\, \varepsilon, t)|^2, \quad d\ loss / d\ output = 2(output - \varepsilon) / size$$

CODE

```
def train_denoiser(steps, seed=0, batch=512, lr=1e-3, every=2000, Xev=None):
    net = d126.init_net([34, 256, 256, 2], seed); opt = d126.Adam(net, lr); acts = ["relu",
        "relu", "linear"]
    rng = np.random.RandomState(seed); log = []
    for s in range(1, steps + 1):
        x0 = d139.ring(batch, rng).astype(np.float64); t = rng.randint(1, len(BETAS) + 1, batch)
        e = rng.standard_normal(x0.shape); xt = np.sqrt(ABAR[t])[:, None] * x0 + np.sqrt(1 -
            ABAR[t])[:, None] * e
        A = d126.forward(net, np.concatenate([xt, time_embed(t)], 1).astype(np.float32), acts)
        d = (A[-1] - e) * 2 / e.size; grads, _ = d126.backward(net, A, acts, d.astype(np.float32))
        opt.step(net, grads)
        if s % every == 0 and Xev is not None:
            log.append((s, eval_eps(eps_net_fn(net), *Xev)))
    return net, log
```

- `rng.randint(1, len(BETAS) + 1, batch)` — a random step from 1 to 1,000 for each point (the upper bound is excluded).
- `np.sqrt(ABAR[t])[:, None] * x0` — ABAR[t] with an array t picks one value per point; [:, None] lines them up with the two coordinates.
- `(A[-1] - e) * 2 / e.size` — the gradient of the mean squared error: 2(guess – target) divided by the number of values.
- `if s % every == 0 and Xev is not None:` — evaluate every 2,000 steps, and only when evaluation data was passed in.
- `eval_eps(eps_net_fn(net), *Xev)` — *Xev unpacks the tuple (x0, t, e) into three arguments.

Segment 3 — Measuring

eval_eps and mode_spread

CODE

```
def eval_eps(eps_fn, x0, t, e):
    xt = np.sqrt(ABAR[t])[:, None] * x0 + np.sqrt(1 - ABAR[t])[:, None] * e; total = 0.0
    for s in np.unique(t):
        k = t == s; total += ((eps_fn(xt[k], s) - e[k]) ** 2).sum()
    return float(total / e.size)
def mode_spread(S, radius=2.0, k=8):
    C = np.c_[np.cos(2 * np.pi * np.arange(k) / k), np.sin(2 * np.pi * np.arange(k) / k)] * radius
    near = ((S[:, None] - C[None]) ** 2).sum(-1).argmin(1)
    return float((S - C[near]).std(0).mean())
```

- `for s in np.unique(t):` — the distinct step numbers, sorted; eps_fn takes one t at a time.
- `k = t == s` — a boolean mask: the triples at this step.
- `C[near]` — fancy indexing: each sample's nearest centre, one row per sample.
- `.std(0).mean()` — the standard deviation of the offsets in x and in y, then their average.

Segment 4 — The experiments

demo_hand: the posterior against a straight-line fit

CODE

```
for t in (2, 50, 300, 900):
    xp = d139.q_sample(np.full(n, x0), t - 1, ABAR, rng); xt = d139.q_step(xp, BETAS[t - 1], rng)
    A = np.c_[np.ones(n), xt]; coef, *_ = np.linalg.lstsq(A, xp, rcond=None); res = (xp - A @
        coef).var()
    c0, ct, var = posterior(t)
    print(f"{t:6d} {c0 * x0:9.5f} {coef[0]:10.5f} {ct:9.5f} {coef[1]:10.5f} {var:11.6f}
        {res:10.6f}")
```

- `np.c_[np.ones(n), xt]` — a column of ones (the intercept) beside x_t: the design matrix of a straight-line fit.
- `coef, *_ = np.linalg.lstsq(A, xp, rcond=None)` — least squares; `*_` collects the other three results, which are not needed.
- `(xp - A @ coef).var()` — the variance left after the fit: the posterior's variance.

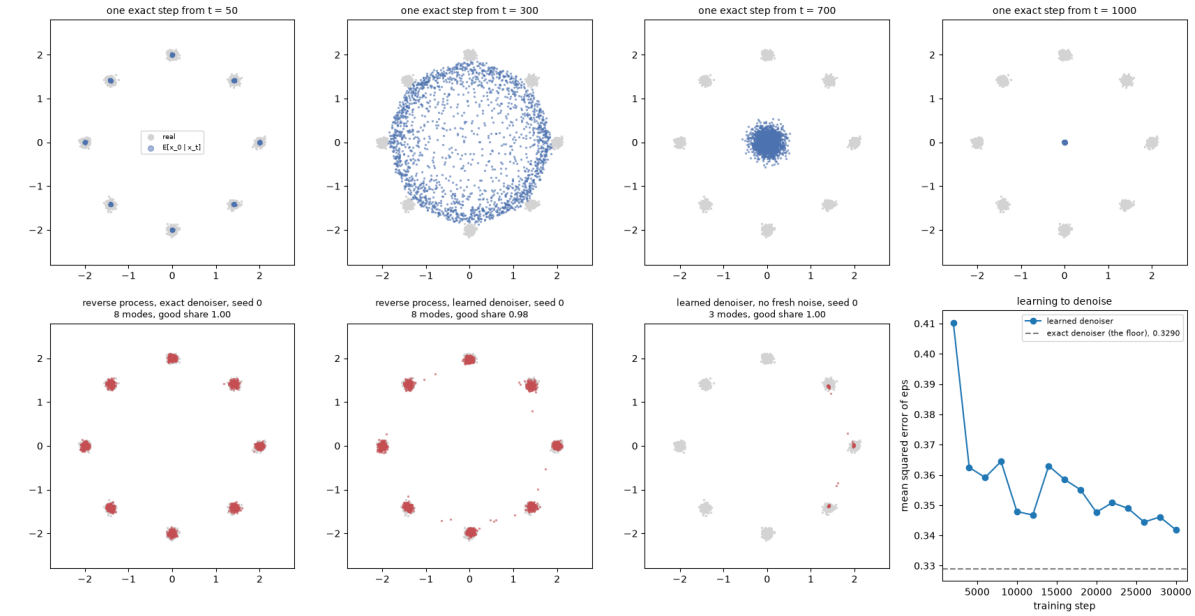
demo_sampling: every denoiser with every noise setting

CODE

```
for name, fn in (("learned", learned), ("exact", exact)):
    for var in ("posterior", "beta", "none"):
        for seed in (0, 1, 2):
            S = reverse_sample(fn, 2000, np.random.RandomState(seed), var); m, g, c =
                d139.mode_stats(S); out[(name, var, seed)] = S
            print(f"{name:>9} {var:>10} {seed:5d} {m:6d} {g:11.3f} {mode_spread(S):7.4f}   {'
                '.join(str(v) for v in c)}")
```

- `for name, fn in (("learned", learned), ("exact", exact)):` — functions stored in a tuple alongside their names.
- `out[(name, var, seed)] = S` — a tuple as a dictionary key: one entry per combination.
- `{name:>9}` — right-aligned in 9 characters.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `range(n, 0, -1)` — count down from n to 1.
- `np.c_[a, b]` — join arrays as columns.
- `A[..., None]` — add a last axis, whatever the leading ones are.
- `coef, *_ = f()` — keep the first result, collect the rest into a throwaway list.
- `lambda x, t: ...` — a small function written inline, here returned from another function.
- `d[(a, b, c)] = v` — a tuple as a dictionary key.

End of Day 140 syntax reference. For the experiments and what they showed, see the Day 140 complete guide.